



Konteyner Temelleri ve Kubernetes'e Giriş

Kubernetes ve Konteyner Dünyası



Bu Konuda

- Konteyner temelleri
- Linux namespace ve cgroup kavramları
- İmaj, registry ve katmanlar
- Konteyner yaşam döngüsü temelleri
- Container runtime'lar
- Kubernetes nedir?
- Kubernetes'e neden ihtiyaç duyulur?

Konteyner Temelleri





Konteyner Nedir?

A container is a standard unit of software that packages up code and all its dependencies... (Docker)

Konteyner, kodu ve tüm bağımlılıklarını paketleyen standart bir yazılım birimidir.

- Konteyner, **izole çalışan bir süreç topluluğudur**, bir sanal makine değildir
- Host kernel'i üzerinde çalışır; izolasyon **Linux namespace + cgroup** mekanizmalarıyla sağlanır
- Aynı **imaj** dan başlatılan konteynerler aynı ikili bileşenleri **host'un kernel'ile paylaşır**



Konteyner ile Sanal Makine Farkı

	Konteyner	Sanal Makine
Sanallaştırılan katman	İşletim sistemi seviyesi (kernel paylaşımı)	Donanım (hypervisor)
İşletim sistemi	Host kernel'i paylaşır	Her VM kendi guest OS'unu çalıştırır
Başlangıç hızı	Saniyeler, süreç başlatma hızında	Dakikalar, donanım önyükleme hızında
Boyut	MB seviyesinde katmanlı imajlar	GB seviyesinde tam disk imajları
İzolasyon	Namespace + cgroup (kernel mekanizmaları)	Hypervisor sınırı (daha güçlü)
Kaynak tüketimi	Yalnızca sürecin ihtiyacı	Guest OS + yardımcı süreçler

- **Aynı kernel paylaşımı** hem hafiflik avantajı hem de izolasyon zayıflığıdır

→ **Pod Güvenliği ve Observability**



Linux Namespace'leri

Namespace, **bir kaynak türünün sürece izole bir görünümünü** sağlayan kernel mekanizmasıdır. Konteyner, bu namespace'lerin bir kombinasyonu ile kurulur. ^[6]

Namespace	İzole ettiği kaynak
PID	Süreç ID'leri
Network	Ağ arayüzleri, IP adresleri, yönlendirme ("routing"), güvenlik duvarı ("firewall", netfilter) kuralları
Mount	Dosya sistemi mount noktaları
UTS	Hostname ve NIS domain adı
IPC	System V IPC, POSIX mesaj kuyrukları ("message queue")
User	Kullanıcı ve grup ID eşlemesi (UID/GID)
cgroup	Süreçlerin gördüğü cgroup kök görünümü

- Linux'ta ayrıca bir Time namespace'i de vardır; konteyner modelinde yukarıdaki 7'si belirleyicidir ^[6]



cgroup'lar

cgroup'lar, **süreçlere tahsis edilen kaynakları kısıtlamak** için kullanılır:

CPU, bellek, disk I/O, PID sayısı vb. ^[18]

Örnek: `docker run --memory=256m --cpus=1` → runtime, limitleri cgroup'a yazar; kernel bunları **uygular** ("enforce"). ^[19]

Controller	Kısıtladığı kaynak
<code>cpu</code>	CPU zamanı paylaşımı
<code>memory</code>	Bellek kullanımı (limit, OOM)
<code>io</code>	Blok/disk I/O
<code>pids</code>	Süreç sayısı
<code>cpuset</code>	CPU/affinity kümesi
<code>hugetlb</code>	Büyük sayfa (hugepage) tahsisi

- **cgroup v1:** her controller **ayrı hiyerarşide** - CPU ve bellek limitleri farklı ağaçlarda yaşar ^[14,18]
- **cgroup v2:** tek **birleşik hiyerarşi** - tüm limitler aynı klasörde: `cpu.max`, `memory.max`, `pids.max` ^[14,18]
- CPU ve bellek limitleri cgroup üzerinden uygulanır; kaynak yönetiminin temelidir ^[2]
→ **Kaynak Yönetimi**

İmaj, Konteyner ve Runtime





İmaj Kavramları

- **İmaj:** Uygulama ve tüm bağımlılıklarını paketleyen, çalıştırılabilir yazılım paketi
- **Konteyner:** Bir imajdan başlatılan çalışan örnek (süreç)
- **Registry:** İmajların saklandığı ve dağıtıldığı servis (ör. Docker Hub, `registry.k8s.io`)^[4]
- **Repository:** Bir imaj serisinin registry içindeki adı (ör. `library/nginx`)

```
[registry host][:port]/repository[:tag] [@digest]  
docker.io/library/nginx:1.27-alpine  
registry.k8s.io/pause:3.5
```



İmaj Katmanları ve Tag/Digest

Karakteristik	Tag	Digest
Değişkenlik	Taşınabilir (aynı tag farklı içeriği gösterebilir)	Değişmez (immutable) ^[4]
İçerik	Sürüm işareti	İçeriğin hash'i (sha256:...)
Kullanım	<code>nginx:1.27</code>	<code>nginx@sha256:1ff6c18f...</code>

- İmaj, **sadece okunabilir katmanlardan** (layers) oluşur; konteyner başlarken üstüne **yazılabilir bir katman** eklenir^[11]
- Katmanlar önbellek paylaşımı sağlar → aynı temel imajı (base image) kullanan konteynerler indirme israfı yapmaz^[11]
- **Digest ile sabitlemek ("pin")** her zaman aynı kodun çalıştırılmasını garanti eder^[4]
→ **Pod Güvenliği ve Observability**



Konteyner Dosya Sistemi Geçicidir

- Konteynerin yazılabilir katmanı, **konteyner yaşam süresiyle sınırlıdır**
- Konteyner silinince (veya node yeniden başlayınca) **tüm yazılan veri kaybolur**

Konteyner dosya sistemi, üstten alta şu katmanlardan oluşur:

Katman	Özellik
Konteyner yazılabilir katman	Geçici; konteyner silinince kaybolur
İmaj katmanları (read-only)	Değişmez; her konteyner aynı içeriği kullanır

- **Volume** ise ayrı ve kalıcı bir depolama alanıdır
→ **Depolama**



Standard Output, Standard Error ve Exit Code'lar

- Konteynerler log'larını **stdout/stderr'e** yazar; runtime bunları toplar
→ Bu akışlar, komut satırı araçlarıyla okunabilir ([Kurulum ve İlk Cluster](#), [Cluster Kurulumu ve Node İşlemleri](#))

Exit code	Anlam
0	Başarılı sonlanma
1 - 124	Uygulama hatası (genel hata 1)
125 - 127	Docker'a özel: 125 daemon hatası, 126 komut çalıştırılmadı, 127 komut bulunamadı ^[10]
128 + N	N numaralı sinyal ile öldürülme ^[9]
137	= 128+9 → SIGKILL (cgroup bellek limiti aşıldığında) → Kaynak Yönetimi ^[8,14,15]
143	= 128+15 → SIGTERM (graceful shutdown isteği) → Pod ve Pod Yaşam Döngüsü ^[8,12]

- Runtime, konteyner exit code'unu durum olarak raporlar



PID 1 Davranışı ve Sinyaller

- Konteyner içindeki ilk süreç (`ENTRYPOINT`) **PID 1** olarak çalışır^[7]
- **PID 1 sonlandığında konteyner sonlanır** (diğer süreçlerin akıbetinden bağımsız)^[13]
- PID 1'e düşen ek görevler:
 - **Sinyalleri almak** (runtime, PID 1 dışındaki süreçlere sinyal iletmez)
 - **Sahipsiz (orphan) süreçleri toplamak** - zombi yönetimi^[7]



ENTRYPOINT, CMD, Command ve Argument ilişkisi

- **ENTRYPOINT** → her zaman çalışan **çalıştırılabilir komut**
- **CMD** → **varsayılan parametreler** (imajı çalıştıran, parametre vermezse)
- Sonuç: konteyner aslında **ENTRYPOINT + args** çalıştırır; `docker run nginx -g 'daemon off;'` gibi komutlar **CMD'yi ezer**^[10]

```
docker run nginx                # ENTRYPOINT + varsayılan CMD
docker run nginx --with-param    # ENTRYPOINT + sağlanan parametre
```



Konteyner Portu ile Dışarı Yayınlanan Port Farkı

Kavram	Anlamı
Konteyner port	Konteynerin kendi network namespace'i içinde dinlediği port (ör. <code>nginx:80</code>)
Yayınlanan port	Host üzerine port eşlemesi (<code>docker run -p 8080:80</code>) - dış dünyadan erişilebilir ^[10]

- Konteyner portu uygulamanın kendi iç görünümüdür; **dışarı erişimi otomatik sağlamaz**
- Konteyner portu tanımlanır
 - **Ağ Modeli ve Uygulama Erişimi**



Container Runtime'lar

- **Container runtime:** Konteynerleri çalıştırmaktan sorumlu yazılım (containerd, CRI-O, Mirantis, Docker Engine) ^[3]
- Runtime'ın başlıca görevleri:
 - İmajı **çekmek** (registry'den) ve açmak (unpack) ^[16]
 - Konteyner için **namespace/cgroup'ları kurmak** ^[13]
 - Süreci **başlatmak** ve yaşam döngüsünü yönetmek (start/stop/delete) ^[13]
- **OCI (Open Container Initiative):** imaj formatını ve runtime davranışını **standartlaştırır** → runtime'lar değiştirilebilir ^[13,16]
- **Katmanlı yapı:** containerd/CRI-O/Docker gibi yüksek seviyeli runtime'lar, **runc** gibi düşük seviyeli OCI runtime'larını kullanır ^[17]

Kubernetes'e Giriř





Kubernetes Nedir?

Kubernetes is a portable, extensible, open source platform for managing containerized workloads and services that facilitate both declarative configuration and automation. ^[1]

Kubernetes, konteynerleştirilmiş iş yüklerini ve servisleri yönetmek için taşınabilir, genişletilebilir, açık kaynak bir platformdur; declarative yapılandırma ve otomasyonu kolaylaştırır.

- **Ad kökeni:** Yunanca "dümenci/kaptan" (helmsman/pilot); K8s kısaltması; Google tarafından 2014'te açık kaynak
- **Ne yapar:** konteynerleri dağıtır, ölçekler, yeniden başlatır; istenen durumu (desired state) gerçeğe yaklaştırır
- **Ne değildir:** bir PaaS değildir; kaynak kodu derlemez veya dağıtmaz; "saf orkestrasyon" değil, sıralı iş akışı yerine sürekli çalışan kontrol döngüleri ("control loop") ^[1]

Kubernetes'e Neden İhtiyaç Duyulur?





Kubernetes'in Çözdüğü Problemler

1. **Dağıtım, yeniden başlatma, ölçekleme** - konteynerleri "el ile" yönetmek ölçeklenmez
2. **Desired state + reconciliation döngüsü** - istenen durumu beyan ederiz; Kubernetes gerçeği ona yaklaştırır
3. **Declarative yönetim** - nasıl değil, **ne** istediğimizi söyleriz
→ **Kubernetes API ve Manifest Okuryazarlığı**
4. **Soyutlama** - container runtime'ı ve altyapı Kubernetes API'si arkasında gizlenir



Desired State ve Reconciliation Döngüsü

A controller tracks ... objects with a `spec` that represents the **desired state**; the controller makes the current state come closer to it. (*Kubernetes docs, "Controllers"*)

Bir controller, `spec` ile istenen durumu temsil eden nesneleri izler ve mevcut durumu ona yaklaştırır.

- Termostat örneği: sıcaklığı ayarlarsın (**desired**), termostat gerçek sıcaklığı (**actual**) ölçer ve aradaki farkı kapatır^[5]
- Kubernetes'in yerleşik ("built-in") controller'ları (ör. Deployment controller) **API server üzerinden** çalışır:

`spec` → controller → API çağrıları → istenen duruma doğru ilerleme → durum raporlanır (`status`)

```
istediğin (spec) → controller → API server → kubelet/runtime → gerçek durum (status)
                        ↑_____ reconcile (tekrar uygula) _____|
```



Kubernetes'in Soyutlama Katmanları

Katman	Ne soyutlanır
Container runtime	containerd / CRI-O / diğer çalışma katmanlarının farkları
Kubelet	Node içindeki pod'ları yönetir; node'a bağımlılığı gizler
Kubernetes API	YAML/JSON manifestlerle kaynakları tanımlar → altyapıdan bağımsız ifade
Bulut entegrasyonu (<code>cloud-controller-manager</code>)	Bulut API'leri (yük dengeleyici ("load balancer"), depolama ("storage")) aynı Kubernetes API'siyle kullanılır

- Sonuç: Minikube'de de bulutta da **aynı manifestler** çalışır
→ **Kubernetes Mimarisi**, **Ekosistem ve Servis Sağlayıcıları**

Kaynaklar





Kubernetes Dokümantasyonu

1. Kubernetes genel bakış: kubernetes.io/docs/concepts/overview/
2. cgroups: kubernetes.io/docs/concepts/architecture/cgroups/
3. Container runtimes: kubernetes.io/docs/setup/production-environment/container-runtimes/
4. Images: kubernetes.io/docs/concepts/containers/images/
5. Controllers: kubernetes.io/docs/concepts/architecture/controller/



Man Sayfaları, Docker, OCI ve Kernel

6. namespaces(7): man7.org/linux/man-pages/man7/namespaces.7.html
7. pid_namespaces(7): man7.org/linux/man-pages/man7/pid_namespaces.7.html
8. signal(7): man7.org/linux/man-pages/man7/signal.7.html
9. bash(1): man7.org/linux/man-pages/man1/bash.1.html
10. Docker `run` referansı: docs.docker.com/engine/reference/run/
11. İmaj katmanları: docs.docker.com/get-started/docker-concepts/building-images/understanding-image-layers/
12. Docker `container stop`: docs.docker.com/reference/cli/docker/container/stop/
13. OCI runtime-spec runtime.md: github.com/opencontainers/runtime-spec/blob/main/runtime.md
14. cgroup v2: www.kernel.org/doc/html/latest/admin-guide/cgroup-v2.html
15. cgroup v1 memory controller: www.kernel.org/doc/html/latest/admin-guide/cgroup-v1/memory.html
16. OCI Image Spec: github.com/opencontainers/image-spec
17. containerd: github.com/containerd/containerd
18. cgroups(7): man7.org/linux/man-pages/man7/cgroups.7.html
19. Docker Kaynak Kısıtlamaları: docs.docker.com/engine/containers/resource_constraints/